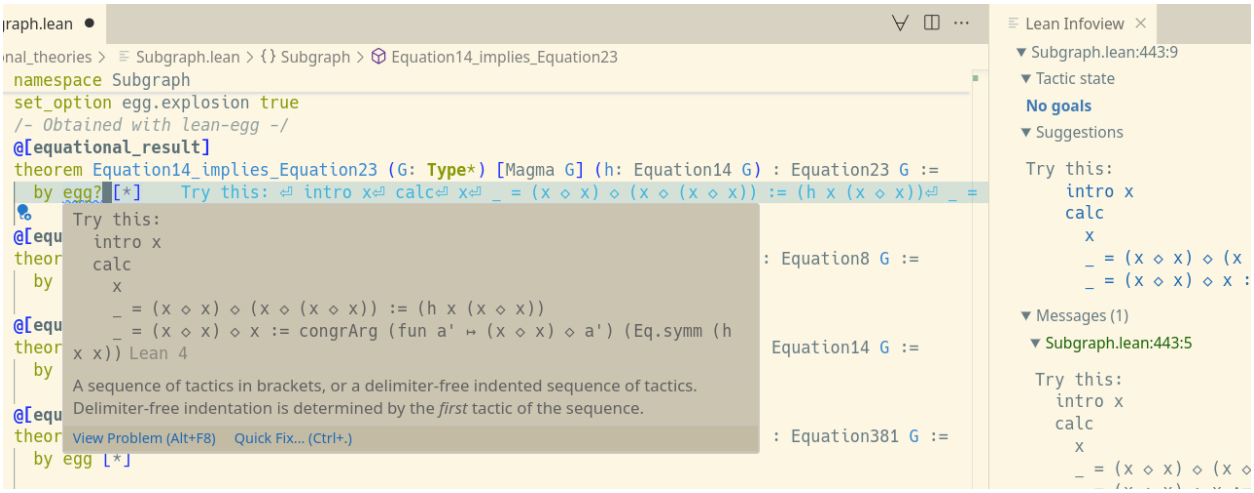


FIGURE 12. Example of the `egg` tactic reconstructing a proof in human-readable form with the help of `calcify` (invoked by the special syntax `egg?`).

In the case of the `egg` tactic, which also reconstructs proofs from *egg* explanations, the proof could be converted into a more human-readable form by using the `calcify`[12] tactic, as shown in Figure 12

7.2.2. *Semi-Automated Counterexample Guidance.* Another use of ATPs has been in a semi-automatic fashion, to find counterexamples. The general strategy was to use ATPs to find counterexamples to implications by building magmas iteratively. If we want to build a counterexample to $l \models l'$, we want to construct a magma where $l$ holds but $l'$ does not. In this method, we iteratively strengthen a construction with additional hypotheses, and use the ATP to check whether these hypotheses are not too strong (to imply $l'$) or unsound (to disallow $l$).

While equational reasoning can also be used in a semi-automatic fashion to prove equations [37], the positive implications in the main implication graph of the project were all simple enough that we did not need a semi-automatic approach for them.

7.3. **ATP usage.** When the project started, contributors had varying degrees of ATP knowledge and skills, with some of us having to start using them from the ground up. With hindsight, several of the project computations could have been approached in better ways, their difficulty diminished due to better ATP expertise. Accordingly, in this section[13] we provide some facts and hints about ATP usage, primarily with the algebraist working with (unsorted) equational theories in mind, based on our experience and available evidence[14] within the ETP. We restrict our attention to *Vampire* and *Prover9-Mace4*, which were the main tools used for exploration of implications and anti-implications along the ETP.

[12] `https://github.com/nomeata/lean-calcify`

[13] Consult the ETP site for a substantially expanded version of these notes.

[14] The timings presented here cannot be taken as benchmarks. Different experiments were executed on different machines, with heterogeneous software and OS environments (Ubuntu, Windows 10...), with varying numbers of parallel processes — both internal and external to the experiment — and, in many cases, under the Linux `nice` command.

7.3.1. *Employing several ATPs.* In general, given a batch of problems of interest, it is useful to employ several ATPs when they complement each other on the batch — i.e., when they can solve different sets of problems. This is often the case with *Vampire* and *Prover9*: e.g., in a 2024 study [2], it is shown that from a batch of around 770 TPTP problems solved with *Vampire* and *Prover9* together (with some restrictions), around 60% are solved by both, 20% are solved only by *Vampire*, and the remaining 20% are solved only by *Prover9*.[15]

Although older and virtually discontinued, *Prover9* is still very useful with equational logic and algebraic problems. Within the ETP we likewise identified several problems that were easier to solve with *Prover9* than with *Vampire*, and even some solved by *Prover9* but not by *Vampire* (with the configurations we tried). The most salient example is the proof that E102744082 implies the injectivity of the right multiplication map, used in the Higman-Neumann side project (see Section 10), which was originally found by *Prover9* in several hours with parameters chosen to produce a big search space; an optimized choice of *Prover9* options lowers the runtime to 0.2s. Upon contact with the *Vampire* development team, after several attempts they were able to provide a *Vampire* configuration (inspired by the *Prover9* optimization) which produces a proof in 3s.

In addition, *Mace4*'s non-SAT algorithm generally makes it faster at finding models than generic SAT-based finite model builders, such as those implemented in *Vampire*. For ETP problems in particular, a well-configured *Mace4* is faster than *Vampire*'s finite model builder in both the task of finding a model of a given size and that of exhausting a size with no models. For example, for E677 models, *Mace4* is able to exhaust size 8 in 1.5s, and to find a model of size 9 in 16s, while *Vampire*'s finite model builder is unable to perform any of these tasks in 10 minutes. By contrast, *Vampire*'s `casc_sat` mode can be quicker at finding *some* model of some size for a given problem.

7.3.2. *Basic usage.* When running an ATP or finite model builder on a nontrivial problem, the two key recommendations are: 1) Conduct several runs with different search-parameter settings, and 2) provide enough computational resources for each search: memory[16], number of processing cores[17], number of user instructions executed, and especially, time. Along the ETP, some proofs or models that could not be found in under 1 second with a given configuration, could be obtained in under (say) 8 seconds without altering the configuration; other implications initially required runs lasting several minutes, or even hours. Once a proof is found by some configuration, a short and quick proof can typically be found by tweaking that configuration.

The configuration of the search parameters is a difficult art that has become more sophisticated as ATPs have grown in complexity. To address difficult problems, we strongly recommend acquiring a solid understanding of the available configuration options and their effects,

[15] In contrast, in said study the E prover solves only a single problem not solved by *Vampire*.

[16] If a memory limit is not specified, *Vampire* will try to use as much RAM as possible, but by default *Prover9-Mace4* sets memory limits which are rather low for current standards, so we advise raising them before starting a long computation.

[17] *Vampire* can make use of as many cores as the user specifies. To take advantage of multicore processors with *Prover9-Mace4*, one can run several terminal or GUI instances. Moreover, each GUI window allows to run *Prover9* and *Mace4* simultaneously on the same problem, and they run on different cores.

enabling the search space to be tailored as closely as possible to the problem at hand. Currently, options allow control over numerous aspects of each proving stage, including search limits, preprocessing steps, inference rules, formulas ordering and weighting, etc.

7.3.3. *Flow control.* *Vampire* is equipped with a user-friendly and powerful standard mode, the CASC mode (or CASC SAT mode for finite model building) that in many situations avoids the need of configuring specific search options for the given problem. This mode invokes a sequence of strategies (different option configurations) with assigned time limits. The time given to each strategy is important: counterintuitively, giving less time to the CASC mode may end up producing a faster proof: there are proofs that can be found when the time limit is set to less than 5% of the time *Vampire* needs to find a proof without the time limit [38]. At least two factors contribute to this effect: the correct strategy is explored sooner because less time is spent on each strategy, and the behaviour of *Vampire*'s default saturation algorithm, the limited resource strategy (LRS) algorithm.

On the other hand, *Prover9* itself does not have the capability of running several schedules in a row, although it lets the user implement some rudimentary strategies through commands called *actions*. Also, a separate program called *FOF-Prover9* includes a preprocessing step that attempts to reduce the problem to independent subproblems, possibly reducing the overall time significantly.

7.3.4. *Input.* Both *Prover9-Mace4* and *Vampire* present pitfalls regarding quantifier scope, which can lead to mistakes for unwary algebraists. Let us illustrate this with *Prover9*, using `*` for the operation $\diamond$. In this ATP, free variables are universally quantified at the outermost level of the whole formula, regardless of its parenthesization. For example, to formulate the property that right multiplications are injective (`y*x = z*x -> y=z`) if and only they are surjective (`exists w w*u=v`), one must include universal quantifiers explicitly on both sides of the binary connective `<->` (quantifiers have higher precedence than `<->`),

```
all x all y all z (y*x = z*x -> y=z) <-> all u all v exists w w*u = v.
```

Omitting the `all` quantifiers would quantify all variables outside the equivalence, while writing the left-hand side as `(all x all y all z y*x = z*x -> y=z)` would amount to $(\forall x, y, z, y \diamond x = z \diamond x) \implies y' = z'$, with $y'$ and $z'$ being free variables, hence universally quantified. We have in fact committed such mistakes over the course of the ETP, thereby giving rise to false proofs of E677 $\models_{\text{fin}}$ E255, our only open implication.

Frequently, it is useful not to run an ATP alone, but to run it in a larger environment permitting multiple calls with different input files, strategies, etc. so that we can provide semi-automated guidance. Integrating the ATP into a computer algebra system further allows to leverage the latter's mathematical capabilities, such as preparing input files with operations from sophisticated algebraic structures. For example, in the ETP we have integrated *Prover9-Mace4* with Python and SAGE, and (among several other applications) used SAGE to access GAP's small groups library to search, via *Mace4*, for translation-invariant countermodels on specific groups (see Section 5.3).

An important factor to consider when generating an input file for proving a conjecture is, for both *Vampire* and *Prover9*, that owing to the way the saturation algorithm operates, the